LABORATORIO 2026

Git y GitHub

Control de versiones con Git y trabajo colaborativo en GitHub



Objetivos de la clase

Al finalizar el encuentro vas a poder:



Interactuar con la terminal con comandos básicos



Conocer Git y qué problema resuelve



Publicar tu primer repositorio en GitHub

Requisitos para esta clase



Git instalado

- Descargar Git Bash → git-scm.com/downloads
- Preinstalado o ejecutar `brew install git`
- Ejecutar `sudo apt install git`



Verificar instalación

Abrir la terminal y ejecutar `git --version`
Se debería imprimir en pantalla algo como: `git version 2.45.0`



Cuenta de GitHub

Crear una cuenta gratuita en github.com

01

SECCIÓN 01

Terminal

Qué es, cómo abrirla y los primeros comandos para moverse sin usar el mouse



¿Cómo abrimos la terminal?

CÓMO ABRIRLA

Windows: `Win + R` → `cmd` → Enter
O buscar «Terminal» en el menú Inicio

macOS: `Cmd + Espacio` → «Terminal»

Linux: `Ctrl + Alt + T`

Windows PowerShell

```
PS C:\Users\alumno> mkdir proyecto
PS C:\Users\alumno> cd proyecto
PS C:\Users\alumno\proyecto>
```

alumno — zsh — 80×24

```
alumno@MacBook ~ % mkdir proyecto
alumno@MacBook ~ % cd proyecto
alumno@MacBook proyecto %
```

¿Qué es la terminal?

DEFINICIÓN

Programa presente en todo sistema operativo que permite dar órdenes a la computadora escribiendo comandos, en lugar de usar el mouse: crear, mover o eliminar archivos y carpetas.

EJEMPLO

En vez de crear una carpeta con clic derecho → Nueva carpeta, se escribe `mkdir proyecto` y Enter: mismo resultado, más rápido y repetible.

Recomendación: usar ”” para los nombres de carpetas

Windows PowerShell

```
PS C:\Users\alumno> mkdir proyecto
PS C:\Users\alumno> cd proyecto
PS C:\Users\alumno\proyecto>
```

alumno — zsh — 80×24

```
alumno@MacBook ~ % mkdir proyecto
alumno@MacBook ~ % cd proyecto
alumno@MacBook proyecto %
```

Rutas (path)

RUTA ABSOLUTA

Comienza **desde la raíz del sistema**. Es la dirección completa del archivo o carpeta.

```
/Users/franco/desktop/proyecto
```

RUTA RELATIVA

Comienza **desde tu ubicación actual**. Usa símbolos para navegar por el sistema de archivos.

`./`	Carpeta actual (donde estás)
`../`	Carpeta padre (subir un nivel)

EJEMPLO

```
/Users/franco/desktop/
├── proyecto/ ← estás aca
│   ├── index.html
│   └── styles.css
└── recursos/
    └── imagen.jpg
```

`./styles.css`
→ Accede a styles.css (misma carpeta)

`../recursos/imagen.jpg`
→ Sube a `desktop/`, luego entra a recursos/

Comandos básicos de la terminal

Algunos commando básicos para interactuar en la terminal:

`dir` Muestra la carpeta actual y sus archivos (en Mac/Linux: `ls` / `pwd`)

`cd [nombre_carpeta]` Entra a una carpeta

`cd ..` Retrocede una carpeta

`mkdir [nombre_carpeta]` Crea una carpeta nueva

`touch [nombre_archivo.extension]` Crea un archivo vacío

`mv [nombre_actual] [nombre_nuevo]` Renombra un archivo

`mv [nombre_archivo] [carpeta_destino]/` Mueve un archivo a otra carpeta

`rm [nombre_archivo.extension]` Elimina un archivo

`rmdir [nombre_carpeta]` Elimina una carpeta vacía

`clear` Limpia la pantalla de la terminal

Práctica 1: tu primera sesión en la terminal

Vamos a interactuar con estos comandos básicos para amigarnos con la terminal.

`dir` Muestra la carpeta actual y sus archivos (en Mac/Linux: `ls` / `pwd`)

`cd [nombre_carpeta]` Entra a una carpeta

`cd ..` Retrocede una carpeta

`mkdir [nombre_carpeta]` Crea una carpeta nueva

`touch [nombre_archivo.extension]` Crea un archivo vacío

`mv [nombre_actual] [nombre_nuevo]` Renombra un archivo

`mv [nombre_archivo] [carpeta_destino]/` Mueve un archivo a otra carpeta

`rm [nombre_archivo.extension]` Elimina un archivo

`rmdir [nombre_carpeta]` Elimina una carpeta vacía

`clear` Limpia la pantalla de la terminal

1. Abrí la terminal (Git Bash en Windows)

2. Creá una carpeta con el nombre “carpeta de prueba”

3. Entrá a la carpeta

4. Creá un archivo con el nombre “archivo.txt”

5. Verificá que el archivo exista en el directorio

6. Comprobá el explorador de archivos

7. Borrá el archivo

8. Regresá a la carpeta anterior

9. Borrá la carpeta creada

10. Verificá los cambios

02

SECCIÓN 02

Git: control de versiones

Qué problema resuelve, cómo funciona el repositorio local y sus comandos básicos



Git y el repositorio local

DEFINICIÓN

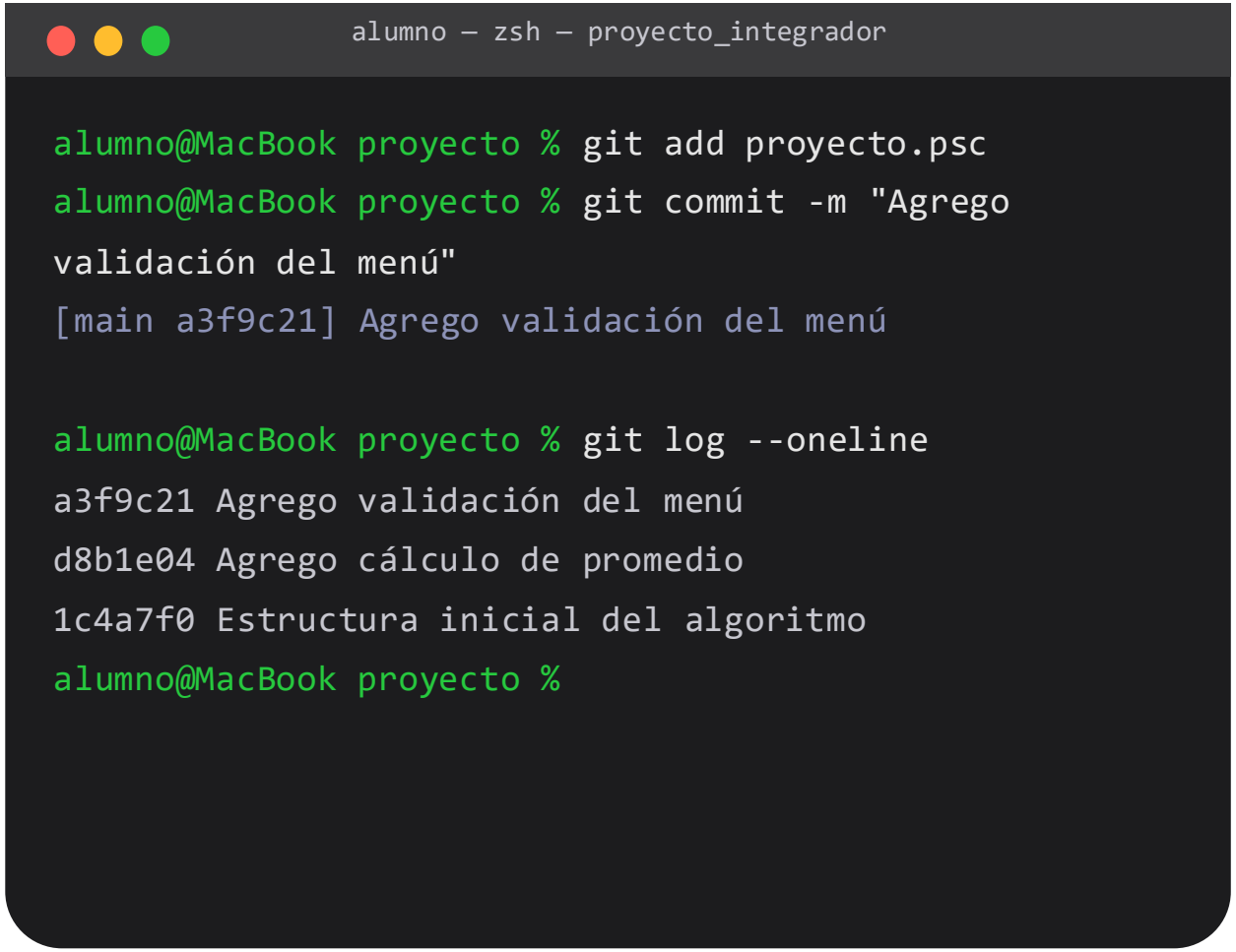
Sistema de control de versiones: lleva un historial ordenado de cada cambio del proyecto (quién, cuándo y qué se modificó), en vez de terminar con archivos tipo `proyecto_v2_FINAL.psc`.

EJEMPLO

En el proyecto integrador de PSEINT: cada vez que sumás una función nueva (ej: la validación del menú) hacés un commit — una "foto" de ese algoritmo, con un mensaje que explica qué cambió.

REPOSITORIO

Carpeta del proyecto que Git versiona: guarda los archivos y todo el historial de commits.



Instalación de Git

Ya debería estar instalado: así lo confirmamos.

`git --version` Confirma que Git esté instalado

→ *Debería devolver algo como: git version 2.45.0*



Configuración inicial: nombre y mail

Se configura una sola vez por máquina: identifica quién hizo cada commit.

```
git config --global user.name [nombre]
```

Define el nombre del autor de los commits

```
git config --global user.email [email]
```

Define el email del autor de los commits

```
git config --global --list
```

Confirma nombre y mail ya configurados

`--global` *aplica esta identidad a todos los repositorios de tu máquina.*

DEMO EN CLASE

Flujo de trabajo: repositorio local

Con el repo iniciado y la identidad configurada, este es el ciclo que se repite en cada avance.

```
git init  Inicia un repositorio en la carpeta actual (una sola vez)
git add [nombre_archivo]  Prepara los cambios para el commit (o "." para todos los archivos)
git commit -m [mensaje]  Guarda una foto del estado actual
git status  Muestra el estado del repositorio
git log / git log --oneline  Muestra el historial de commits

git push  Sube los commits a GitHub — lo vemos en la próxima sección
```

DEMO EN CLASE

Práctica 2: tus primeros commits

PASOS A SEGUIR

1. Configurá tu identidad: `user.name` y `user.email`
2. Iniciar el repositorio con `git init`
3. Crear un archivo de texto de prueba (con contenido)
4. Primer commit: `git add` + `git commit -m`
5. Editar ese archivo y hacer un segundo commit
6. Verificá los commits con `git log` y/o `git log --oneline`

```
# configuración inicial (única vez)
git config --global user.name "Tu Nombre"
git config --global user.email "tu@mail.com"

# repo local
git init
git add archivo.txt # agregarle contenido
git commit -m "Primer commit”

# después de editar el archivo
git add archivo.txt
git commit -m "Segundo commit"
```

03

SECCIÓN 03

GitHub: repositorios remotos

El repositorio remoto: cómo publicar y compartir lo que ya versionamos en local



GitHub y el repositorio remoto

DEFINICIÓN

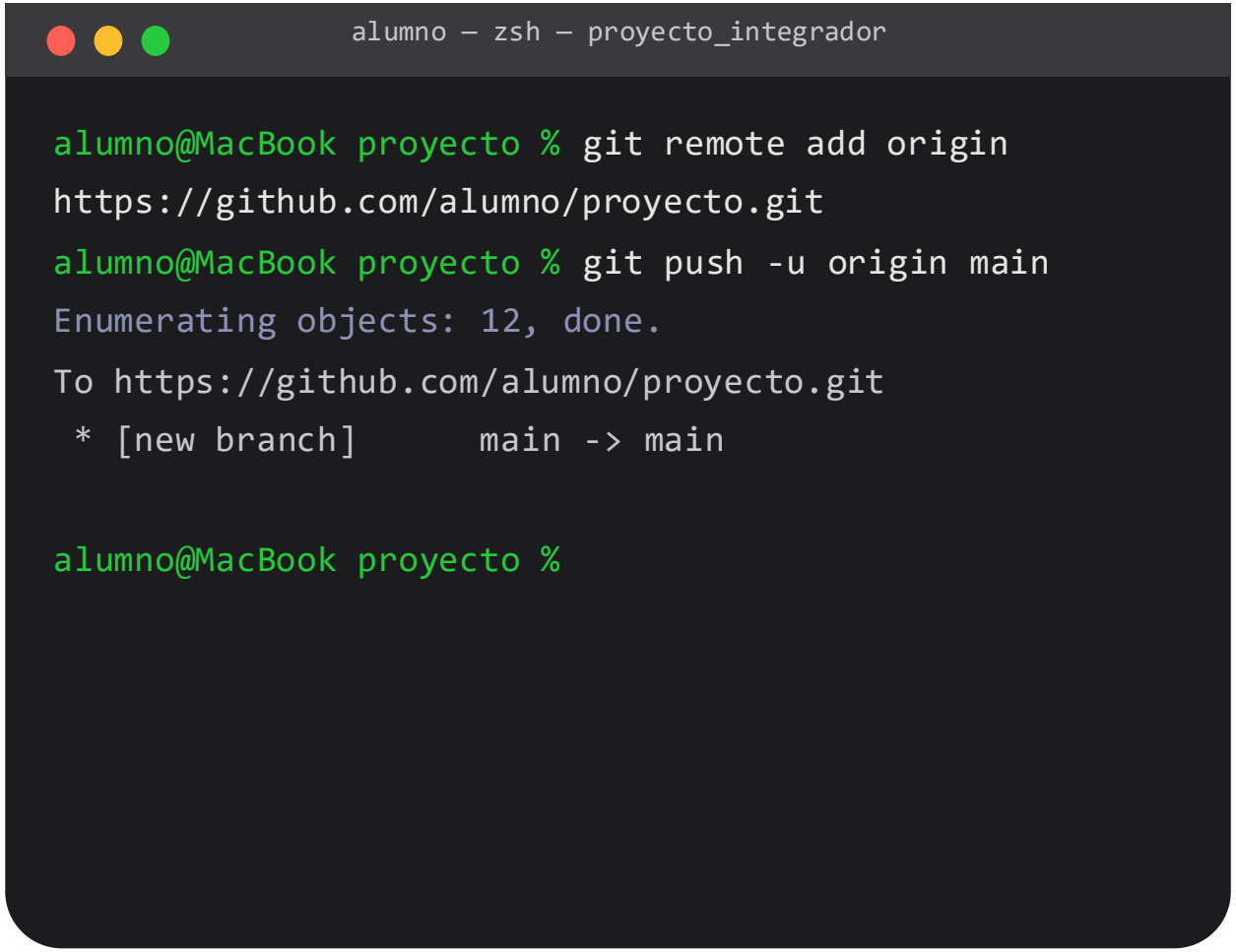
Servicio en la nube donde alojar repositorios Git. A diferencia del repo local, este vive en un servidor: accesible desde cualquier computadora y compartible con otras personas.

Al hacer `git push`, el historial de commits que tengo en mi computadora (local) queda disponible también en GitHub (remoto), visible para los compañeros de trabajo.

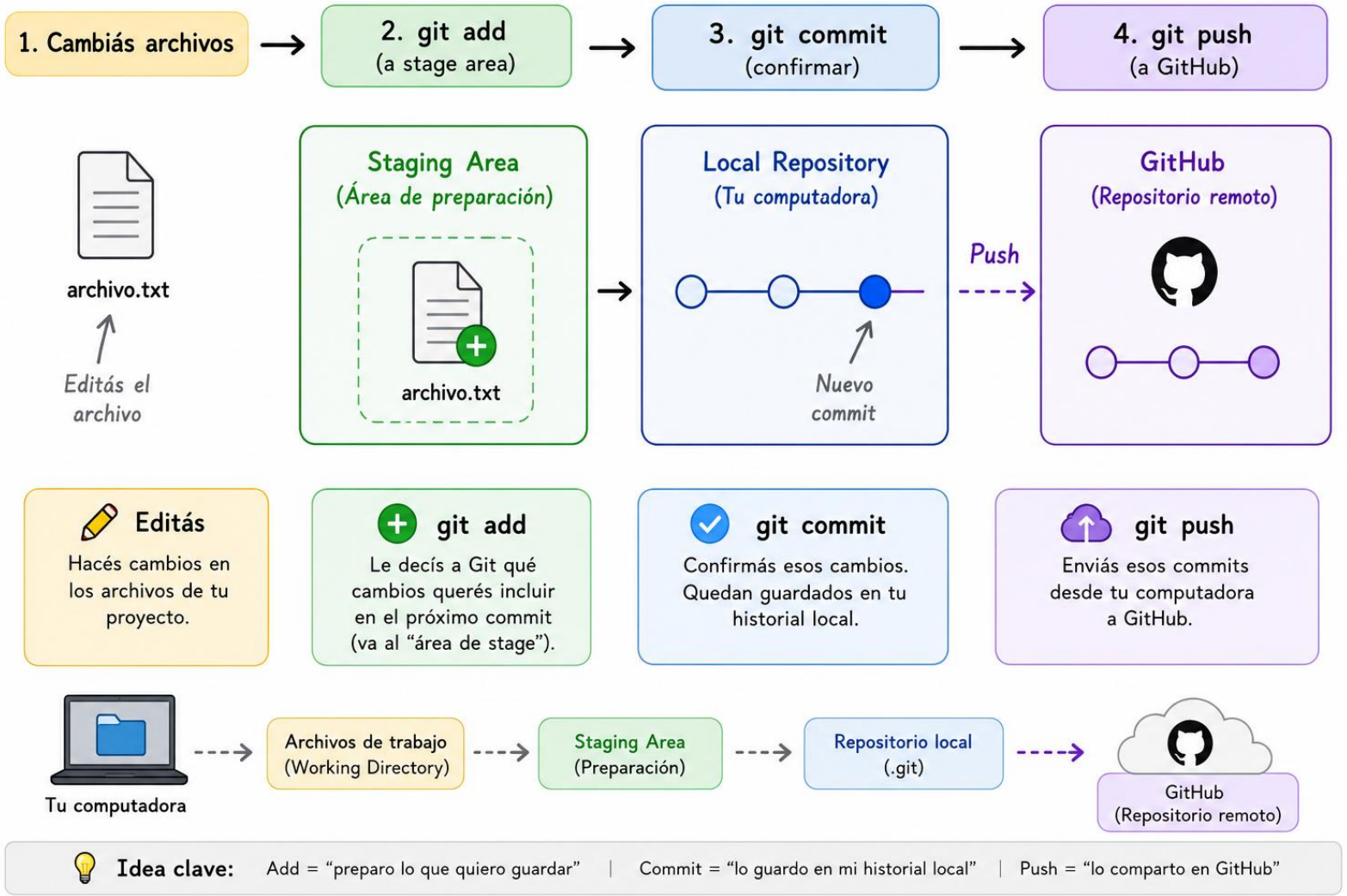
Aclaración: GitHub no es un repositorio: es la plataforma que aloja tus repositorios.

OTROS SERVICIOS

Existen otros servicios como GitLab y Bitbucket, pero Github es el más utilizado.



Flujo básico de Git



Recorrido por la interfaz de GitHub

Demostración en vivo navegando la pantalla — todavía no hace falta escribir nada.

- Perfil de usuario y foto: van a compartir este perfil con docentes y futuros empleadores, vale la pena completarlo
- Panel principal: repositorios y actividad reciente
- Dentro de un repositorio: archivos, commits, ramas y colaboradores

Recomendación: activar verificación en dos pasos (`Settings → Password and authentication → Two-factor authentication`)

DEMO EN CLASE

Crear el primer repositorio en GitHub

Demostración paso a paso

- Botón `New`
- Nombre del repo: normalmente el mismo que la carpeta del proyecto (podría ser cualquiera)
- Visibilidad: `Public`
- No inicializar con README (lo agregaremos luego nosotros)
- Agregar colaboradores: `Settings → Collaborators → Add people` → compañero/a de grupo y docente

`Public` = cualquiera puede verlo.

`Private` = solo vos y quienes invites como colaboradores.

DEMO EN CLASE

Vincular el repo local al remoto y primer push

Se conecta con la carpeta proyecto-laboratorio de la Sección 2, que ya tiene 2 commits.

GitHub muestra los comandos exactos para viuncular tu prepositorio:

```
1. git remote add origin [url]
2. git branch -M main
3. git push -u origin main
```

Al terminar, refrescar GitHub para ver los commits y archivos pusheados.

Cierre conceptual: lo que versionaban localmente ahora es visible en la nube.

DEMO EN CLASE

Push y Pull

Con el repo ya visible en pantalla, es el momento de formalizar los conceptos.

- `push` (Local → Remoto) = subir los commits locales al remoto. Se usa después de cada commit (o grupo de commits) que quieran compartir.

- `pull` (Remoto → Local) = traer al local los commits que hay en el remoto que yo todavía no tengo. Se usa cuando alguien más hizo cambios, o al trabajar desde otra computadora

Flujo de trabajo en equipo: pull → editar → add → commit → push

Recomendación: Verificar el estado de los archivos antes de pushear con git status

Clonar un repositorio

Demostración: el docente crea un repo de prueba y los estudiantes lo clonan.

- `git clone [url]` — descarga una copia completa del repositorio remoto
- `cd nombre-del-repo` — entra a la carpeta recién clonada
- Diferencia clave: `clone` se usa la primera vez, cuando el repo remoto ya existe — a diferencia de `init` + `pull`, deja el origin configurado en un solo paso
- El docente edita un archivo desde GitHub (editor web → commit)
- Los estudiantes corren `git pull origin main` y ven el cambio aparecer: cierra el ciclo entre push y pull, entre personas

DEMO EN CLASE

Comandos básicos de GitHub

Se necesita una cuenta de GitHub creada antes de la clase para poder completar la práctica.

```
git remote add origin [url]   Conecta el repo local con uno remoto en GitHub
git branch -M main            Renombra la rama principal a main
git push -u origin main       Sube los commits locales al repositorio remoto
git clone [url]               Descarga una copia completa de un repo remoto
git pull origin main          Trae y aplica los cambios del remoto al local
```

Cierre: lo que viene

Lo de hoy cubre el 80% del uso diario de Git y GitHub. Todavía quedan estas herramientas:

- **Branches** — trabajar en paralelo sin pisar el código de otros
- **Pull requests** — proponer cambios y revisarlos antes de incorporarlos
- **Conflictos y merge** — qué pasa cuando dos personas editan la misma línea
- **.gitignore** — qué archivos no queremos versionar (claves, node_modules, etc.)

04 SECCIÓN 04

Práctica final

De la carpeta vacía al primer repositorio publicado en GitHub



Práctica 3: Proyecto Integrador a GitHub

PASOS A SEGUIR

1. **Responsable de grupo**: crea el repo en GitHub, lo hace público y agrega colaboradores (compañeros)
2. **Responsable de grupo**: sube su carpeta local
3. **Colaboradores**: clonan el repo y editan un solo archivo cada uno para evitar conflictos y luego commitean.
4. **Un colaborador** pushea primero; el resto del grupo espera
5. **Los demás** "pullean" para traer el nuevo cambio
6. **Opcional**: luego pushean de a uno, repitiendo el ciclo en orden

```
# responsable
git remote add origin [url]
git push -u origin main

# colaboradores
git clone [url]
git add .
git commit -m [mensaje]
git push origin main

# el resto del grupo
git pull origin main
```



¿Preguntas?